

## Tarea Corta 2 – Colas: Atención de Emergencias

Esta tarea consiste en la programación, modificación y utilización de las siguientes estructuras de datos:

1. Cola enlazada (**LinkedQueue**)
2. Tipo de dato abstracto Cola de prioridad (**PriorityQueue**)
3. Cola de prioridad enlazada (**LinkedPriorityQueue**)

La idea general de esta tarea es tomar la implementación de cola enlazada vista en clase (**LinkedQueue**) y reutilizarla para crear una estructura que implementa el tipo de dato abstracto *cola de prioridad* y utilizarla en un programa de atención de pacientes que ingresan a emergencias médicas.

### Parte 1 - Cola de prioridad

Este es un tipo de dato abstracto que permite extraer los elementos en un orden determinado por una prioridad asignada a cada uno. Mientras que en una cola normal el orden de extracción es FIFO, en una cola de prioridad el primer elemento a ser extraído es el que tenga mayor prioridad. Comúnmente, la prioridad se representa con un número entero no negativo donde la mayor prioridad la tiene el valor 0, entre mayor el número, menos prioridad.

Escriba la clase abstracta **PriorityQueue**. Esta clase debe ser genérica para el tipo de elemento (**E**) que se quiere almacenar. Puede eliminarse el constructor de copia y la operación de asignación. A continuación, se muestra la lista de operaciones con las que debe contar la clase y su descripción.

1. **void insert(E element, int priority)**  
Inserta un elemento en la cola con la prioridad indicada.
2. **E min()**  
Retorna el elemento con menor prioridad en la cola. Si hay varios con la misma prioridad retorna el que será eliminado de primero al invocar **removeMin()**.
3. **E removeMin()**  
Elimina y retorna el elemento con menor prioridad en la cola. Si se invoca justo después del método **min()**, debe dar el mismo elemento como resultado.
4. **void clear()**  
Limpia todos los contenidos de la cola de prioridad.
5. **int getSize()**  
Retorna la cantidad de elementos que se encuentran en la cola actualmente.
6. **bool isEmpty()**  
Retorna verdadero si la cantidad de elementos en la cola es cero. De otro modo retorna falso.
7. **void print()**  
Muestra en pantalla los contenidos de la cola.

## Parte 2 - Arreglo de prioridades con colas enlazadas

Una forma “inocente” de implementar una cola de prioridad es utilizando un arreglo de colas enlazadas, donde cada cola contiene los elementos que tienen una misma prioridad. Esto es equivalente a decir que vamos a tener una cola por cada posible prioridad.

Esta implementación tendría un límite máximo de prioridades, por lo que no sería apropiada para escenarios donde la cantidad de prioridades es incierta o puede incurrir en desperdicio de memoria si la cantidad de prioridades posible es grande.

A pesar de eso, la operación de inserción en esta estructura sería muy eficiente porque se utilizaría la prioridad del elemento a insertar como índice para seleccionar la cola respectiva dentro del arreglo y ejecutar la operación **enqueue** de la cola enlazada.

Las operaciones para obtener y eliminar el mínimo requieren una búsqueda secuencial de la primera cola que no esté vacía y luego efectuar la operación **frontValue** o **dequeue**, respectivamente. Esto representa un ciclo que puede degradar estas operaciones, sobre todo si la cantidad de prioridades definidas es alta.

Algo parecido sucede con la operación **clear**, ya que implica limpiar todas colas del arreglo. Para evitar este mismo problema en las operaciones **isEmpty** y **getSize**, se puede utilizar un atributo **size** que se actualice al realizar las operaciones de inserción y borrado.

Ejemplo: cola de prioridad con seis prioridades (0-5).

Se crea un arreglo de 6 colas enlazadas y se realizan las siguientes inserciones (elemento-prioridad): BLB-1, GND-5, FRD-5, LGL-0, MRG-1, AWD-3, GYH-0, ARG-1.

La cola tiene los siguientes contenidos:

|   |     |     |     |
|---|-----|-----|-----|
| 0 | LGL | GYH |     |
| 1 | BLB | MRG | ARG |
| 2 |     |     |     |
| 3 | AWD |     |     |
| 4 |     |     |     |
| 5 | GND | FRD |     |

Si se ejecuta la operación **min**, se obtiene el resultado “LGL”. Al ejecutar la operación **removeMin** repetidas veces hasta que la cola está vacía, se obtienen los elementos en este orden: LGL, GYH, BLB, MRG, ARG, AWD, GND, FRD.

Implemente la clase **LinkedPriorityQueue**. La clase debe heredar de la clase abstracta **PriorityQueue** y también debe ser genérica.

### Atributos:

1. **queues:** puntero al arreglo de colas enlazadas.
2. **priorities:** entero con la cantidad máxima de prioridades a utilizar.
3. **size:** entero con la cantidad de elementos que se encuentran en la estructura.

### Métodos:

#### 1. **LinkedPriorityQueue(int priorities)**

El constructor recibe la cantidad de prioridades que va a utilizar la estructura y crea el arreglo de colas enlazadas en memoria dinámica. Inicializa los demás atributos. Puede usar 20 como cantidad de prioridades por defecto. Debe lanzar error si la cantidad de prioridades no es positiva.

Para crear un arreglo de colas en memoria dinámica puede usar la siguiente línea:

```
queues = new LinkedQueue<E>[priorities];
```

#### 2. **~LinkedPriorityQueue()**

El destructor retorna la memoria del arreglo de colas solicitado por el constructor.

#### 3. **void insert(E element, int priority)**

Agrega el elemento en la cola indicada por el parámetro **priority**. Debe lanzar un error si el número de prioridad no es válido.

Para utilizar una cola del arreglo, simplemente se utiliza la prioridad como índice y se utilizan el método que se desee. Por ejemplo:

```
queues[priority].enqueue(element);
```

#### 4. **E min()**

Retorna el elemento que se encuentra en el frente de la primera cola no vacía. Lanza error si la cola está vacía.

#### 5. **E removeMin()**

Elimina y retorna el elemento que se encuentra en el frente de la primera cola no vacía. Lanza error si la cola está vacía.

#### 6. **void clear()**

Elimina todos los elementos de la cola de prioridad.

#### 7. **int getSize()**

Retorna la cantidad de elementos que encuentran actualmente en la cola de prioridad.

#### 8. **bool isEmpty()**

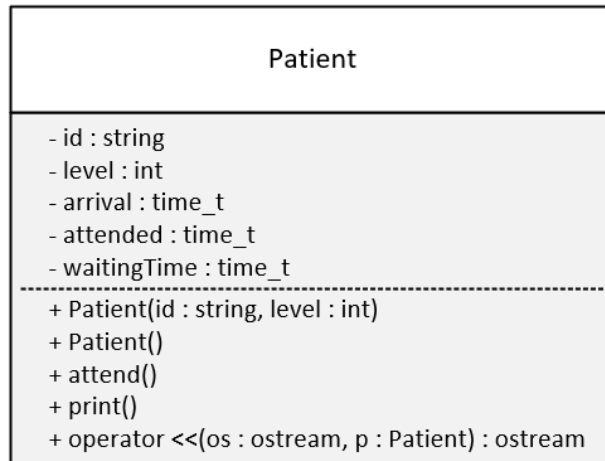
Retorna verdadero si la cola está vacía, de otro modo retorna falso.

#### 9. **void print()**

Imprime los contenidos de la cola en pantalla. Puede imprimirse cada una de las colas con su respectivo índice.

### Parte 3 – Lógica y programa principal

Para modelar las funcionalidades del programa se va a utilizar una clase de lógica para representar los datos de un paciente que ingresa a ser atendido por una emergencia médica (**Patient**). Esta clase tiene un atributo tipo **string** para guardar la identificación de la persona, un entero para el nivel de emergencia y tres atributos tipo **time\_t** (biblioteca **time.h**) para guardar la hora de llegada, la hora de atención y el tiempo de espera hasta ser atendido.



El atributo que representa el nivel de emergencia tendrá el siguiente significado, según su valor:

1. Azul – Resucitación
2. Rojo – Emergencia
3. Amarillo – Urgente
4. Verde – Menos urgente
5. Blanco – No urgente

La clase debe tener un constructor que reciba como parámetro el id y el nivel, la hora de llegada se inicializa con la hora actual al momento de invocar el constructor. También debe crearse un constructor por defecto sin parámetros, de otro modo las colas enlazadas no podrán crear el nodo centinela cuando **E = Patient**.

La clase tiene un método llamado **attend** que se encarga de asignar la hora de atención con la hora del momento en que se invoca el método y también calcular el tiempo de atención restando a la hora de atención la hora de llegada.

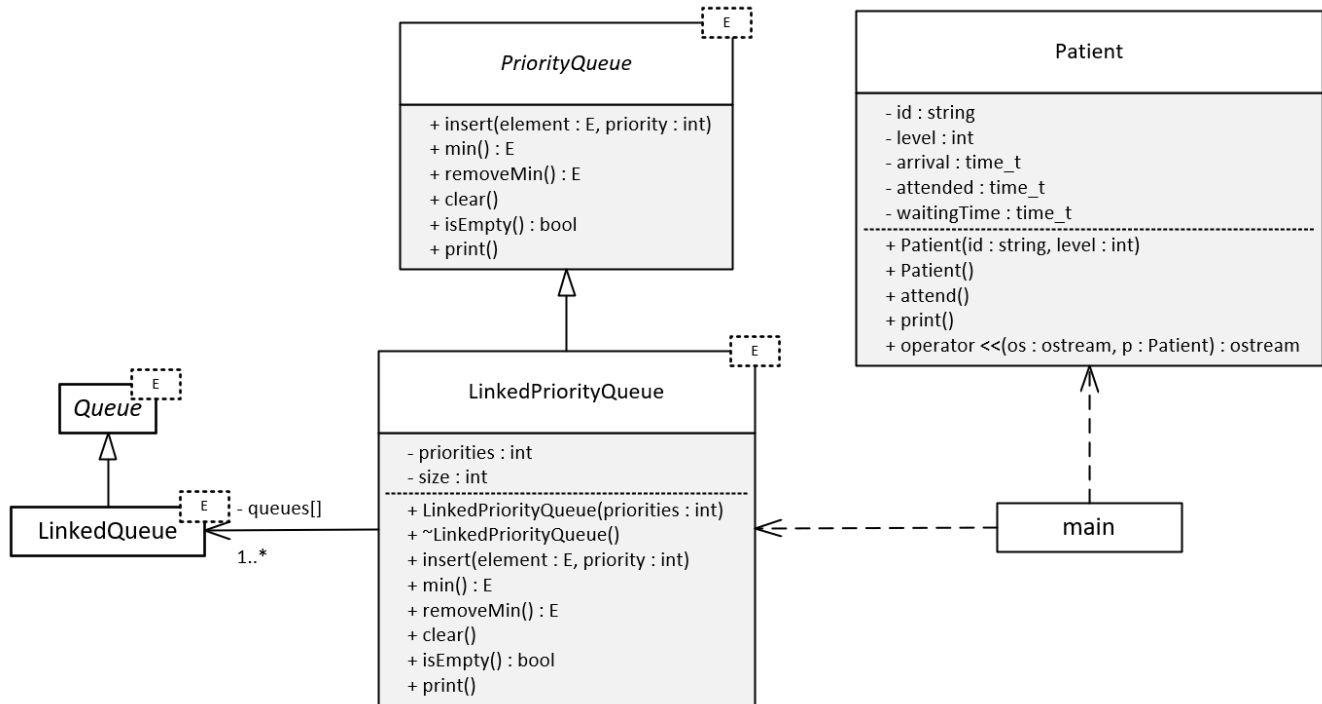
Finalmente debe implementarse un método para imprimir todos los detalles del paciente en pantalla y también sobrescribir el operador de extracción (<<) para poder utilizar el método **print** de la cola de prioridad.

El método **print** de **Patient** debe producir una salida similar a la siguiente:

```
ID: 1233545
CONDICIÓN: 3 AMARILLO - Urgente
LLEGADA: 2025-11-1 18:39:20
ATENDIDO: 2025-11-1 18:39:32
ESPERA: 12 s
```

El operador de extracción debe producir una salida similar a la siguiente: (Id: 1233545, Nivel: 3)

A continuación, se muestra el diagrama de clases que debe seguirse para la implementación del programa. El programa principal hace uso de la clase **Patient** y de una cola de prioridad **LinkedPriorityQueue**. Se muestra que la clase **LinkedPriorityQueue** hereda de la clase abstracta **PriorityQueue** y que contiene un arreglo de **LinkedQueue**, clase que a su vez hereda de **Queue**.



El programa principal debe contar con una cola de prioridad **LinkedPriorityQueue<Patient>** con cinco prioridades para guardar los pacientes que ingresan a emergencia según la clasificación del nivel de emergencia que se explicó anteriormente.

Al iniciar programa se mantiene en un ciclo principal que muestra los contenidos de la cola de prioridad (con el método **print**) y las operaciones que se pueden realizar hasta que el usuario elija la opción para salir.

TRIAGE - Cola de espera:

```
0: [ ]
1: [ ]
2: [ ]
3: [ ]
4: [ ]
```

1. Ingresar paciente
2. Atender paciente
3. Salir

Aunque las prioridades que se utilizan para clasificar la emergencia van de 1 a 5, la cola de prioridad utiliza las prioridades de 0 a 4.

Al utilizar la opción para ingresar paciente se solicita el ID del paciente y el nivel de emergencia (condición) para ingresarlo en la cola de prioridad y regresar al menú principal.

```
Ingresar paciente
ID: 122550336
CONDICIÓN (1-Azul, 2-Rojo, 3-Amarillo, 4-Verde, 5-Blanco): 3

TRIAGE - Cola de espera:
0: [ ]
1: [ ]
2: [ (Id: 122550336, Nivel: 3) ]
3: [ ]
4: [ ]

1. Ingresar paciente
2. Atender paciente
3. Salir
```

Cada vez que se ingresa un paciente, se ve reflejado en los contenidos de la cola.

```
TRIAGE - Cola de espera:
0: [ (Id: 101110222, Nivel: 1) ]
1: [ (Id: 355541654, Nivel: 2) (Id: 123354555, Nivel: 2) ]
2: [ (Id: 122550336, Nivel: 3) (Id: 16545555, Nivel: 3) (Id: 115156464, Nivel: 3) ]
3: [ (Id: 255553312, Nivel: 4) ]
4: [ (Id: 655554441, Nivel: 5) (Id: 355256656, Nivel: 5) ]
```

Al utilizar la opción para atender paciente se extrae el siguiente paciente de la cola de prioridad. Al extraerlo debe ejecutarse el método **attend** para que se actualice la hora de atención y el tiempo de espera. También se muestran sus datos con el método **print**.

```
PACIENTE EN ATENCIÓN:
ID: 101110222
CONDICIÓN: 1 AZUL - Resucitación
LLEGADA: 2025-11-1 19:0:55
ATENDIDO: 2025-11-1 19:2:44
ESPERA: 109 s
```

Si la cola se encuentra vacía, debe mostrarse un mensaje adecuado.

Al elegir la opción para salir, debe retornarse cualquier memoria dinámica que se haya utilizado durante la ejecución del programa y terminar.

## Otros detalles del programa principal

1. El programa debe utilizar manejo de errores con la estructura *try/catch* para detectar cualquier situación anómala.
2. El programa debe rechazar cualquier entrada no válida escrita por el usuario y volverla a solicitar.
3. Los menús de la interfaz de usuario deben usar opciones numéricas como las mostradas.
4. Se recomienda utilizar solamente la función *getline* para leer las opciones del usuario.
5. El programa, al finalizar, debe eliminar los objetos que se hayan creado en memoria dinámica.
6. Divida el programa principal en subrutinas.

¡Buena suerte!

Entrega: Sección de evaluaciones del TEC-Digital.

Formato: Archivo comprimido ZIP con el proyecto completo. Corrobore que se incluyen todos los archivos necesarios para compilar y ejecutar el proyecto.